

Low-Risk Arbitrage and Market-Neutral Tools for Polymarket and Manifold (March 2026)

Overview

This report surveys concrete, low-frequency tools and strategies that can be deployed by mid-March 2026 to improve a prediction market bot's risk-adjusted returns on Polymarket and Manifold, with optional Kalshi integration. It emphasizes cross-platform arbitrage infrastructure, market-neutral trades, Claude Code-friendly tooling, cheap external data, and robust risk controls under a tight API budget and 1–7 day holding periods.[1][2]

The recommendations assume an existing Python-based stack with daily scheduled runs, a database of trades, and LLM-based probability calibration, as in the user's poly-what repository and exit manager design. The action plan at the end ranks 10 specific items by impact, feasibility, and time-to-deploy.

1. Cross-Platform Arbitrage Infrastructure

1.1 Public REST market data across platforms

Polymarket exposes public market data via its Gamma/markets REST endpoints without authentication, returning question text, prices, and token IDs, suitable for periodic polling. Manifold similarly offers unauthenticated endpoints such as /markets and /market/{id} for market listings and details, with authenticated /bet only needed for execution. Kalshi provides public market data over its trade-api/v2/markets and events endpoints, with a quick-start guide showing how to list open markets and retrieve titles, tickers, and yes-prices using plain requests.[1][3][4][5][2][6]

1.2 WebSocket vs. REST for low-frequency strategies

Polymarket offers an order-book WebSocket (<wss://ws-subscriptions-clob.polymarket.com>) with channels for book depth and trade feeds, but this is paywalled behind a premium subscription (~\$99/month), which exceeds a very small tooling budget. Kalshi and Manifold also support WebSockets for real-time data; tutorials and SDKs show how to subscribe to order-book and trade updates in Python, but these are most useful for latency-sensitive or intraday strategies rather than 1–7 day holds. For low-frequency, low-risk trading, REST polling every 1–5 minutes is sufficient to detect durable mispricings while keeping infrastructure simple and within cost constraints.[3][4][2][7][8]

1.3 Existing arbitrage bots and tooling

Polymarket documents official open-source client SDKs in TypeScript, Python, and Rust that cover the full CLOB (market data and order management), including the `py-clob-client` Python library. The Polymarket GitHub organization also hosts an agents repository for AI-driven agents trading on the CLOB, which can serve as reference architecture for integrating LLM signals with order execution. For cross-platform arb, there is an open-source Rust "Polymarket–Kalshi arbitrage bot" that scans and executes arbitrage opportunities across the two venues; although written in Rust, it is useful for seeing how others match markets, calculate spreads, and manage two-legged execution.[9][10][11][12][13]

A January 2026 Reddit writeup describes a library that started as a Polymarket–Kalshi arbitrage scanner and has been extended to direct order execution on both platforms, aiming for near-atomic execution on both sides of a spread. This shows that cross-platform arb is actively being automated in 2026, but also highlights that the main bottleneck is safe execution rather than just identifying spreads.[14]

1.4 Cross-platform analytics and dashboards

Several third-party dashboards aggregate prediction market data. Polymarket Analytics provides cross-platform search and real-time analytics for Polymarket and Kalshi, including the ability to compare markets and monitor trade activity for arbitrage-like opportunities. Community-maintained dashboards on Dune Analytics and Bitquery,

collected in the "Awesome-Polymarket-Tools" list, offer volume, open interest, and user analytics that can be repurposed as monitoring backends. On GitHub, the polymarket-subgraph-analytics repository demonstrates how to build comprehensive dashboards by querying multiple Polymarket subgraphs, including an orderbook subgraph with spreads, depth, and last trade prices.[15][16][17]

2. Market-Neutral and Low-Risk Strategies

2.1 Within-platform statistical arbitrage ($YES + NO < 1$)

A detailed January 2026 article deconstructs five Polymarket arbitrage styles, starting with the classic within-market arbitrage when the sum of YES and NO prices for a binary contract falls below 1. The principle is to buy both sides when $YES + NO < 1$ (after accounting for fees), locking in a riskless profit at resolution since the payoff must sum to 1. The same source emphasizes that these opportunities often appear during volatility or panic, especially in illiquid markets or when large orders temporarily distort the order book.[18]

2.2 Cross-platform arbitrage of equivalent events

The same arbitrage article describes cross-platform arbitrage, where equivalent events listed on different venues diverge in price because of differences in trader base or rules. For example, a YES contract at 45 cents on Polymarket and an economically equivalent NO contract at 52 cents on Kalshi yields a locked spread if both oracles resolve consistently. However, the article warns that differences in resolution criteria or oracles can lead to different settlement outcomes, making it crucial to carefully compare fine print and resolution policies before treating markets as equivalent.[18]

2.3 Information and resolution-based arbitrage

The Polymarket arbitrage taxonomy also highlights "information preemption" arbitrage, where traders use small timing advantages in live data (sports scores, vote tallies, etc.) to enter trades before prices fully adjust on-chain. For a low-frequency bot, this can be adapted to resolution-based arbitrage in markets that will clearly resolve within 24 hours, focusing on situations where off-chain information (e.g., a

completed game score) is final but the on-chain market has not yet fully converged. Another style discussed is volatility-based arbitrage, where traders use probability distributions to ensure bounded losses even in adverse scenarios, effectively constructing markets so typical outcomes yield profit while tail events are near break-even.[19][18]

2.4 Market making and spread capture

Market-making arbitrage is another category: by placing limit orders above and below current prices in low-liquidity markets, traders can earn the bid–ask spread while providing liquidity. Manifold hosts an open-source market-making bot that computes exponential moving averages and variances of market probabilities and then posts symmetric limit orders around the current price to profit from volatility. While pure market-making can be riskier and more capital-intensive, its logic demonstrates how to use volatility and partial fills to create market-neutral or near-neutral PnL profiles over many small trades.[20][18]

2.5 LLM- and model-driven prediction bots as case studies

A March 2025 case study describes a Manifold bot built in a weekend using GPT-4o and o1 consensus, which ranked fourth in a Manifold prediction league and reportedly 10x'd its play-money bankroll by focusing on mispriced newly created markets. The bot only placed small bets when both models agreed within a narrow band and differed significantly from the market probability, which is analogous to the user's existing LLM calibration divergence strategy. This demonstrates that cautious, low-frequency mispricing exploitation can scale in practice without requiring high-frequency execution.[21]

3. Tooling for Claude Code and Python Automation

3.1 Polymarket official Python SDKs

Polymarket's documentation lists official client SDKs in Python (`py-clob-client`) and TypeScript (`@polymarket/clob-client`), as well as a Rust client, all open source and documented with examples. The Python client shows how to construct a `ClobClient`, derive or create L2 API credentials from a private key, and post signed limit orders (including GTC orders) given token IDs from the Gamma markets API. Builder-specific signing SDKs (`py_builder_signing_sdk`) are also available if building as part of Polymarket's Builder Program, simplifying EIP-712 signing and attribution.[10][9][1]

3.2 Manifold API wrappers and bot frameworks

The `manifoldpy` library provides Python bindings for the Manifold API, including an `APIWrapper` that stores an API key and exposes methods such as `make_bet` for placing bets on a given contract and outcome. This wrapper offloads authentication and request details, letting strategy code interact with Manifold markets at a higher level of abstraction. Additionally, "Sketchy's Bot Framework" (described on a Manifold blog post) offers a Manifold API client that manages WebSocket connections, an abstraction over core entities (`Bet`, `Market`, etc.), and a strategy framework with a core event loop.[22][7]

This framework is designed for custom strategy plug-ins, letting developers focus on strategy logic while it manages low-level WebSocket and API details, which makes it compatible with asynchronous Claude Code workflows that trigger strategy evaluations on new market events.[7]

3.3 Kalshi Python SDKs and real-time examples

Kalshi maintains an official Python SDK (`kalshi_python`) with typed models and methods to list markets, retrieve candlestick data, and fetch trades for specific tickers. Documentation includes examples of configuring an authenticated `KalshiClient` with an API key and private key, and using methods such as `get_market_candlesticks` and `get_trades` with limit and cursor-based pagination. Separate tutorials and videos demonstrate how to subscribe to Kalshi order-book updates using WebSockets in Python, building a real-time order-book

watcher for observing price action and identifying order-book inefficiencies.[23][4][8]

3.4 Pre-built dashboards and analytics repos

GitHub hosts multiple Polymarket-focused analytics projects. `polymarket-subgraph-analytics` shows how to query various Polymarket subgraphs (markets, PnL, activity, orderbook, names) and combine them into a dashboard-ready, human-readable dataset with key metrics like spreads, depth, and last trade price. Other repositories such as `polymarket-intelligence` and `polymarket-dashboard` implement real-time tracking dashboards, often using Next.js or similar frameworks, featuring live price charts, whale order tracking, and news feeds. Community-curated lists like "Awesome-Polymarket-Tools" catalog additional dashboards (Dune, Bitquery, commercial tools like PolymarketDash) that provide volume, OI, trader analytics, and alerting.[24][25][16][26][15]

3.5 n8n workflows and low-code integrations

An official n8n template demonstrates how to collect historical price data from Polymarket Up/Down serial markets into Supabase, including series discovery, event collection, token ID resolution, and batched historical price retrieval. Another example from late 2025 describes an n8n workflow chaining GPT and Nansen to automatically analyze breaking events relevant to Polymarket, combining news scraping and on-chain flow analysis to flag mispricing opportunities and push alerts to messaging channels. These workflows align well with a Claude Code–centric stack, where n8n handles event-driven orchestration and storage while Claude Code scripts implement strategy logic.[27][28]

4. Data Sources for Edge (Free or Cheap)

4.1 Real-time sports scores

API-SPORTS offers a broad sports data platform with real-time scores, covering over 2000 competitions across major sports; it has both a free plan and paid tiers starting at roughly 10 dollars per month. Independent reviews of free sports APIs highlight API-SPORTS as providing live updates every 15 seconds and a generous free tier with

about 100 requests per day, which is adequate for low-frequency monitoring of a small set of games relevant to open prediction markets. For more focused football or cricket data, alternatives like Sportmonks provide a "free forever" plan with 180 calls per hour per endpoint, though the user's needs may be met entirely by API-SPORTS given the budget.[29][19]

4.2 Political prediction and forecasting aggregators

Metaculus provides a rich API using the MetaculusApi class in its open-source forecasting-tools library, including methods to fetch specific questions by URL or ID, retrieve all open questions in tournaments, and query for resolved questions under complex filters. The same tools expose benchmark questions and functions to post predictions and comments, making it feasible to use Metaculus community forecasts as priors or sanity checks for similar prediction markets on Polymarket or Manifold. A separate metac-bot-template repository shows how to set up bots using Metaculus tokens and external LLM/search providers, providing working examples for tournament forecasting that can be adapted for cross-platform comparisons.[30][31]

Other political aggregators like FiveThirtyEight and PredictIt have historically provided data feeds or APIs, but in 2026 much of this data must often be accessed via scrapers or third-party libraries, and licensing constraints may apply; Metaculus therefore stands out as a clean, programmable source for forecast distributions.[31][30]

4.3 Social sentiment from Twitter alternatives

Bluesky provides an AT Protocol API that can be accessed in Python via the atproto client, which supports login using a handle and app password, and lets users fetch post threads and extract author, timestamp, and text fields. This enables basic per-account or per-hashtag monitoring, such as tracking mentions of specific candidates or teams and measuring posting frequency or engagement as lightweight sentiment features.[32]

Mastodon exposes both REST and streaming APIs, and the Mastodon.py library supports the full API including real-time streaming via helper methods like stream_public and stream_user,

using a `StreamListener` interface for updates. Tutorials show how to use either the official streaming API or simple HTTP streaming with requests to track public timelines, hashtags, or accounts, printing username, content, and timestamps for each new status in real time. These open, free APIs are well-suited for low-budget, low-frequency sentiment monitoring around political and sports events.[33][34][35][36]

5. Risk Controls and Portfolio Management

5.1 Core risk management practices for algos

General algorithmic trading risk guidance emphasizes several pillars: smart position sizing, diversification, stop-loss systems, maximum drawdown controls, volatility-based adjustments, and model risk controls. Recommended practices include limiting risk per trade (e.g., fixed fractional or volatility-scaled sizing), capping exposure to individual markets or segments, and enforcing automated drawdown limits that pause trading after a specified loss. Diversification is particularly important: research summarized by risk-management providers notes that international or cross-asset diversification can lower volatility while preserving or improving risk-adjusted returns, which in this context translates to not concentrating solely on crypto or politics markets.[37]

5.2 Volatility-based position sizing

Volatility-based position sizing adjusts trade sizes inversely to recent volatility, often using the Average True Range (ATR) metric in traditional markets, to trade smaller in choppy regimes and larger when conditions are calmer. Practical implementations in algorithmic trading scale position size as $\text{Monetary Risk} \div (\text{ATR} \times \text{point value})$, with guidance to update volatility estimates regularly and to adjust stop distances accordingly. In prediction markets, an analogous approach could use rolling standard deviation of implied probabilities or historical returns to tune bet sizes, thereby stabilizing PnL and reducing the impact of sudden price jumps.[37]

5.3 Correlation-aware portfolio construction

Academic work on cryptocurrency portfolios demonstrates that clustering assets by historical correlations and then applying portfolio optimization (e.g., via PyPortfolioOpt) can produce portfolios that capture hidden structure while reducing risk. Such frameworks use correlation networks and community detection (e.g., Louvain algorithm) to group highly correlated assets, then allocate across clusters to balance exposure. Although focused on crypto, similar methods can be applied across prediction market segments—grouping markets into categories such as US politics, global geopolitics, macro/interest rates, sports, and crypto sentiment, then limiting exposure per cluster.[38]

5.4 Low-frequency strategy considerations

Discussions of low-frequency strategies in quant communities stress the importance of aggregating higher-frequency signals into stable, lower-frequency features, employing rolling windows to derive thresholds, and dynamically adjusting them rather than hard-coding static values. For prediction markets, this suggests using rolling measures of win rate, edge, and realized volatility to update divergence thresholds and position sizes over time instead of keeping fixed thresholds tuned to a short backtest. Integrating regime filters (e.g., risk-on vs. risk-off, event-heavy vs. quiet weeks) can further reduce tail risk and avoid overtrading during noisy periods.[39][37]

5.5 Black-swan and circuit-breaker protections

Risk-management best practices include maximum drawdown controls, which halt trading when cumulative losses exceed a predefined threshold, and model risk controls that prevent algorithms from behaving unexpectedly due to bugs or out-of-sample conditions. For algorithmic traders, recommended safeguards include per-day loss limits, max number of losing trades in a row, and automated disabling of new entries when vol spikes beyond expected ranges. These controls are especially suitable for low-frequency prediction-market bots, as they sacrifice little expected edge while significantly reducing tail risk.[37]

6. Ranked Action Plan (8–12 Items)

The following action plan is tailored to a Python- and Claude Code-based prediction-market bot with existing LLM calibration, daily schedulers, and paper-trading infrastructure. Each item lists implementation steps, expected profit impact (qualitative), and estimated time to deploy.

6.1 Build a cross-platform REST price monitor and discrepancy logger

Implementation steps:

- Use Polymarket's public markets endpoint, Manifold's /markets API, and Kalshi's trade-api/v2/markets to periodically fetch active markets resolving within 1–14 days, via a Python script scheduled every 1–5 minutes.[4][5][6][3]
- Maintain a small mapping file (YAML/JSON) of semantically equivalent markets across platforms (e.g., S&P up/down, election outcomes), curated manually at first.
- For each mapped pair or set, compute implied probabilities and check conditions like: cross-platform spread > 5–10 percentage points, or Polymarket YES + Kalshi NO < 1 after fees.[18]
- Log candidate opportunities to the database and send alerts via email/Telegram or n8n, including both platform prices, spread size, and time stamp.[28][27]

Expected profit impact: Medium. This reduces missed opportunities by systematizing cross-platform mispricing detection, adding 1–3 high-quality trades per week once mappings are fleshed out, with minimal risk because execution can remain manual or semi-automated at first.[17][18]

Time to deploy: <1 day for a minimal REST-based monitor over a handful of mapped markets.

6.2 Implement within-Polymarket sure-arbitrage ($YES + NO < 1$) scanner

Implementation steps:

- Extend the existing Polymarket data-fetching code to compute, for each binary market, the sum of best-bid YES and best-bid NO prices from the CLOB or the last trade prices.[1][3]
- Filter for markets where $YES + NO < 1$ minus a safety buffer for platform fees and slippage (e.g., threshold at 0.97–0.98), as described in the Polymarket arbitrage article.[18]
- Implement a checklist: minimum liquidity, time to resolution (e.g., < 7 days), clean resolution criteria, and no known oracle quirks.
- At first, only record these cases in a dashboard or log and optionally enter both sides manually; later, automate GTC limit orders via py-clob-client with strict size caps.[10][18]

Expected profit impact: Medium–high. These are structurally positive-EV or guaranteed-profit trades if sized modestly and executed carefully; even a few per month can materially outperform generic directional trades at similar risk.

Time to deploy: 1–3 days to integrate the scan, logging, and a basic manual execution playbook.

6.3 Add portfolio-level caps by market cluster and per-trade sizing rules

Implementation steps:

- Tag existing and new markets in the database with categories (e.g., US politics, global geopolitics, macro/Fed, crypto prices, sports) and store notional exposure per category.[38]
- Implement hard limits, such as maximum 20–25% of bankroll in any single category and 5–10% in any single market, using account-equity snapshots from Manifold and Polymarket APIs. [5][3][37]
- Add a correlation-aware layer later: export historical price series (using n8n + Supabase for Polymarket, direct API for Manifold) and compute rolling correlations or cluster

assignments for categories, borrowing ideas from crypto portfolio clustering.[28][38]

Expected profit impact: Medium. This does not directly boost raw returns but significantly improves risk-adjusted PnL and drawdown behavior by preventing overconcentration in correlated themes like US elections or crypto macro.

Time to deploy: <1 day for simple caps using existing schema; 1–3 days if adding basic correlation analysis.

6.4 Implement volatility-based position sizing overlay

Implementation steps:

- For each market, compute recent volatility of implied probabilities or log-returns using historical prices (e.g., last 7–14 days), stored via your existing historical data workflows or the n8n Polymarket-Supabase template.[28][37]
- Translate the ATR-style formula from traditional markets to prediction markets, scaling bet size inversely to volatility so that more volatile markets receive smaller stakes for the same risk budget.[37]
- Integrate this into existing entry logic so that LLM-calibrated edges determine direction but volatility determines size, capped by category and per-market limits.

Expected profit impact: Medium. Volatility-scaling tends to stabilize returns and reduce the incidence of large single-trade losses, thereby increasing Sharpe ratio even if average edge per trade remains similar.[38][37]

Time to deploy: 1–3 days, mainly to build and validate the volatility estimator and test position-size outputs across historical trades.

6.5 Introduce account-level circuit breakers and daily loss limits

Implementation steps:

- Define numeric thresholds such as maximum daily realized loss (e.g., 2–3% of bankroll), maximum intraday drawdown, and

- maximum number of consecutive losing trades.[37]
- Add a daily risk-monitoring script that summarizes realized and unrealized PnL from the DB and platform APIs; when thresholds are exceeded, set a flag in configuration that prevents new entries while allowing exits.[3][5]
- Log all circuit-breaker triggers with reasons and context to support post-mortem analysis and threshold tuning.

Expected profit impact: Medium. These controls primarily protect against black-swan sequences, fat-finger errors, or structural breaks (e.g., model failures or sudden rule changes on a platform), significantly reducing risk of catastrophic drawdowns at negligible cost to expected long-run returns.[37]

Time to deploy: <1 day for a basic implementation using existing trade and PnL logging.

6.6 Leverage n8n workflows for Polymarket data and event-driven alerts

Implementation steps:

- Deploy or adapt the n8n template that extracts historical Polymarket Up/Down market data into Supabase, giving you structured historical prices for backtests, volatility estimation, and signal research.[28]
- Create a second n8n workflow that, on a schedule or event trigger, pulls current Polymarket markets relevant to your themes and calls Claude Code (or another LLM) to summarize notable moves or anomalies, similar to the GPT+Nansen event-analysis workflow.[27]
- Connect these workflows to Slack/Telegram or a simple web dashboard for human-in-the-loop oversight, especially for resolution-based or information-preemption opportunities.

Expected profit impact: Medium. Better historical data and automated anomaly alerts help refine entry rules and highlight mispricings or liquidity cliffs, enabling a few higher-quality discretionary or semi-automated trades per week.

Time to deploy: 1–3 days, assuming prior familiarity with n8n and Supabase.

6.7 Integrate Metaculus forecasts as external priors for selected markets

Implementation steps:

- Use the forecasting-tools library’s MetaculusApi to fetch questions and community forecasts for a curated list of political and macro questions that map cleanly onto Polymarket or Manifold markets.[30]
- For each candidate market, compute the difference between Metaculus community median and platform-implied probabilities; require agreement between LLM estimates and Metaculus before entering larger position sizes.
- Log these features in your DB to analyze over time whether Metaculus-aligned trades exhibit higher win rates or lower variance.

Expected profit impact: Low–medium. This adds an extra sanity-check layer and may reduce bad trades where LLMs disagree with expert forecasters, improving win rate at the cost of fewer trades.

Time to deploy: 1–3 days to wire, map a small set of overlapping questions, and integrate into the signal stack.

6.8 Add sports resolution-based trades powered by API-SPORTS

Implementation steps:

- Identify a subset of Polymarket (or other venue) sports markets where outcomes map directly to final scores or match results covered by API-SPORTS.[19][29]
- Use the API-SPORTS free tier or entry-level paid plan to monitor relevant matches in near real-time, focusing on late-game or immediately post-game windows where the outcome is decided but market prices have not fully adjusted.[19]
- Implement a small module that, for such markets, flags when off-chain data indicates a final result, then either alerts you to

close positions or (with caution) enter a final arb/hedge trade if prices are stale.

Expected profit impact: Low–medium. These opportunities may be sporadic but are highly skewed toward low risk when carefully constrained to clear, final outcomes and liquid markets.

Time to deploy: 1–3 days, including testing API usage and mapping between sports events and market identifiers.

6.9 Incorporate basic social-sentiment monitors (Mastodon/Bluesky)

Implementation steps:

- Use the `atproto` client to log in to Bluesky and periodically fetch posts from specific political or macro commentators, extracting timestamps, text, and engagement counts.[32]
- Set up Mastodon.py with a `StreamListener` to monitor selected instances, hashtags, or accounts related to elections, macro events, or crypto, counting posts and measuring spikes in activity.[36][33]
- Start with simple heuristics (e.g., a sudden surge in posts around a particular issue) that raise a human-review alert; later, experiment with basic sentiment scoring if bandwidth allows.

Expected profit impact: Low initially. The main benefit is qualitative context and early warning around regime shifts or major news that could impact clusters of markets, helping avoid holding concentrated thematic risk into volatile events.[39][37]

Time to deploy: About 1 week for robust, production-quality monitors, though simple prototypes can be built faster.

6.10 Gradual adoption of Manifold bot frameworks for play-money experimentation

Implementation steps:

- Experiment with `manifoldpy` and "Sketchy's Bot Framework" to run strategies on Manifold's play-money markets, using its

WebSocket-powered event loop and classes for markets and bets.[22][7]

- Port a simplified version of your existing LLM-divergence strategy into the framework, and use Manifold's API documentation and case-study bots as reference for best practices.[5][21]
- Evaluate which parts of this architecture (e.g., event loop, strategy abstraction) can be adopted into your Polymarket/real-money stack without increasing operational complexity.

Expected profit impact: Medium, but mostly via skill and tooling improvements rather than immediate PnL. Prototyping on Manifold with play money de-risks strategy changes and new market-neutral tactics before deploying real capital on Polymarket or Kalshi.

Time to deploy: About 1 week for a fully integrated, tested workflow given existing experience with Python and LLM integration.

Conclusion

By March 15, 2026, a low-risk bot operator can realistically deploy a cross-platform REST-based arbitrage monitor, within-Polymarket sure-arb detection, portfolio caps, volatility-based sizing, and robust circuit breakers, all within a modest API budget. Complementary steps—integrating n8n data workflows, Metaculus priors, sports APIs, and social sentiment—provide incremental edge and risk awareness while staying within Python- and Claude Code-friendly tooling. Leveraging Manifold play-money frameworks for experimentation ensures that new ideas are tested safely before touching real USDC on Polymarket or regulated venues like Kalshi.[4][7][21][36][30][3][5][19][18][28][37]

References

1. [Overview - Polymarket Documentation](#) - Polymarket's CLOB (Central Limit Order Book) is a hybrid-decentralized trading system — offchain ord...
2. [Best Prediction Market APIs for Developers and Traders](#) - Manifold Markets API. Manifold Markets operates differently—it uses play money (Mana) rather than re...

3. [Quickstart - Polymarket Documentation](#) - All data endpoints are public — no API key or authentication needed. Use the markets endpoint to fin...
4. [Markets - Kalshi's API Documentation](#) - Get Markets List and discover markets on Kalshi. A market represents a specific binary outcome withi...
5. [Manifold API](#) - Our API was historically hosted on <https://manifold.markets/api> , but we recently moved to give the ...
6. [Quick Start: Market Data \(No SDK\) - Kalshi's API Documentation](#) - This guide will walk you through accessing Kalshi's public market data endpoints without authenticat...
7. [Sketchy's Bot Framework - Manifold](#) - A Manifold API client that handles WebSocket connections and API requests. · Classes that abstract c...
8. [Kalshi Order Book Watcher: Websockets For Real-Time Data in Python](#) - ... Kalshi order book updates using Python and websockets. 0:00 Intro ... Analyze Prediction Market ...
9. [Clients & SDKs - Polymarket Documentation](#) - Polymarket provides official open-source clients in TypeScript, Python, and Rust. All three support ...
10. [py-clob-client 0.24.0 - PyPI](#) - Python client for the Polymarket CLOB. Full API documentation can be found here. Installation. pip i...
11. [ryan26craf/polymarket-kalshi-arbitrage-bot - GitHub](#) - A high-performance arbitrage bot written in Rust that identifies and executes arbitrage opportunitie...
12. [polymarket-kalshi-arbitrage-trading-bot · GitHub Topics](#) - Polymarket Kalshi Arbitrage Bot: Rust trading bot for cross-platform arbitrage between Polymarket (P...
13. [Polymarket - GitHub](#) - Polymarket. Polymarket has 95 repositories available. Follow their code on GitHub.
14. [Automating the Prediction Market Arb: Programmatically capturing ...](#) - The goal is to move from "monitoring a dashboard" to "atomic-ish execution" where you can hit both l...
15. [PaulieB14/polymarket-subgraph-analytics - GitHub](#) - NEW: With the Polymarket Names subgraph and the advanced Orderbook subgraphs, you can now build comp...

16. [harish-garg/Awesome-Polymarket-Tools - GitHub](#) - Dune Analytics - Polymarket Dashboards - Multiple community-created dashboards for volume tracking, ...
17. [Polymarket Analytics – Prediction Market Data, Analytics, and ...](#) - Real-time prediction market data, analytics, and dashboards for Polymarket. Track top traders and ac...
18. [Deconstructing Polymarket's Five Arbitrage Strategies: How Can ...](#) - Real-world case study: How did top traders earn millions of dollars on the polymarket? 1▯ Arbitrage...
19. [API-Sports - Restful API for Sports data](#) - API-SPORTS Real-time sports data. Take your project to the next level with our sports data set desig...
20. [manifoldmarkets/market-maker: A market-making bot for Manifold's ...](#) - A market-making bot for Manifold. Create limit orders via the Manifold API to provide liquidity, and...
21. [Building a diamond Manifold prediction bot in a weekend - 2025/04/01](#). I built a prediction market bot in a few hours that ranked fourth in Manifold's Platinum...
22. [vluzko/manifoldpy: Python tools for working with Manifold Markets](#) - Tools for analyzing Manifold Markets data. Currently has bindings for their API, and code for comput...
23. [Prediction Markets with the Kalshi API \(Python Tutorial\) - YouTube](#) - 0:00 Kalshi - Why is this interesting to me? 1:35 Comparison with Polymarket 3:56 Semantic matching ...
24. [buddies2705/polymarket-dashboard - GitHub](#) - Polymarket Dashboard. A Next.js dashboard application for displaying Polymarket prediction market da...
25. [luuisotorres/polymarket-intelligence - GitHub](#) - Polymarket Intelligence. Real-time market tracking dashboard for Polymarket with live price charts, ...
26. [PolymarketDash: Professional trader analytics and real-time ...](#) - A Polymarket analytics and trader monitoring dashboard supporting leaderboard insights, historical p...
27. [n8n Automates Analysis of Polymarket Events with GPT and Nansen](#) - n8n integrates GPT and Nansen to automate Polymarket event analysis, reducing cognitive costs by 90%...
28. [Collect historical price data from Polymarket Up/Down markets into ...](#) - 4.6

29. [12 Best Free Sports API Options for Developers in 2025](#) - Discover the top 12 free sports API providers for 2025. Get access to reliable sports data for your ...
30. [Metaculus/forecasting-tools: A framework for building a AI ... - GitHub](#) - This repository contains forecasting and research tools built with Python and Streamlit. The project...
31. [Metaculus/metac-bot-template - GitHub](#) - 30min Video Tutorial. This tutorial shows you how to set up our template bot so you can start foreca...
32. [Looking up posts in the Bluesky API - alexwlchan](#) - Install the atproto package, construct a client with your username/password, then call the `get_po...
33. [Streaming — Mastodon.py 2.1.4 documentation](#) - Streaming . These functions allow access to the streaming API. Since Mastodon v4.2.0 the support fo...
34. [Mastodon.py — Mastodon.py 2.1.4 documentation](#) - A simple Python wrapper for that API, provided as a single Python module. [Mastodon.py](#) aims to implem...
35. [How to Stream from the Mastodon API via HTTP in Python](#) - Watch an instance; Track certain hashtags; Monitor an account. Note: It's worth mentioning that stre...
36. [pypi-mastodon-py@2.1.1 • tessel • Registry](#) - Supports the complete Mastodon API (v4.4.3) including authentication, posting, media handling, real...
37. [7 Risk Management Strategies for Algorithmic Trading - NURP](#) - Learn seven essential risk management strategies for algorithmic trading, including position sizing,...
38. [Optimising cryptocurrency portfolios through stable clustering ... - arXiv](#) - In this paper, we develop a technical framework that uses historical data on daily closing prices an...
39. [Augmenting low frequency features/signals for a higher ... - Reddit](#) - A way to somehow smartly aggregate and derive lower frequency signals from higher frequency ones. Wh...